



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

DIPARTIMENTO DI INFORMATICA - SCIENZA E INGEGNERIA - DISI

CORSO DI LAUREA IN
INGEGNERIA INFORMATICA

WEB SCRAPING MEDIANTE LA LIBRERIA SELENIUM

Tesi di laurea in INGEGNERIA INFORMATICA

Relatore

Prof. Ing. Paolo Bellavista

Presentata da

Alberto Catellani

Sessione Ottobre 2024

Anno Accademico 2023/2024

Sommario

Sommario.....	2
Ringraziamenti.....	4
Premessa.....	5
Introduzione.....	6
Capitolo 1: il potere dei dati.....	8
1.1 Web scraping.....	8
1.2 Vantaggi.....	9
1.3 Svantaggi.....	9
1.4 Implicazioni Etiche e Legalità.....	10
1.5 Scopo della tesi.....	11
1.6 Le librerie di web scraping.....	12
Capitolo 2: architetture e librerie.....	13
2.1 Identificazione degli elementi.....	13
2.2 Un problema ricorrente.....	14
2.3 La libreria Selenium.....	14
2.3.1 Installazione di Selenium.....	16
2.3.1.1 Installazione del driver.....	16
2.3.1.2 Selenium Driver Manager.....	17
2.3.1.3 Selenium Browser Manager.....	18
2.3.2 Flusso di lavoro di Selenium.....	20
2.3.3 Strategie di caricamento della risorsa.....	22
2.3.4 Accesso agli elementi.....	24
2.3.5 Selenium Grid.....	27
2.3.6 BiDirectional API.....	29
2.4 La libreria Puppeteer.....	31
2.4.1 Installazione.....	31
2.4.2 Modalità headless.....	31
2.4.3 Gestione del browser.....	32
2.4.4 Architettura.....	32
2.4.5 Lancio del browser.....	33
2.4.6 Chiusura del browser.....	33
2.4.7 Connessione ad un browser già aperto.....	33
2.4.8 Un template classico.....	34
2.4.9 Interazione con gli elementi della pagina.....	34
2.4.10 Velocità di Puppeteer.....	35

2.5	La libreria Cheerio.....	36
2.5.1	Architettura.....	36
2.5.2	Il caricamento degli elementi.....	37
2.5.3	Selezione degli elementi.....	38
2.5.4	Attraversamento del DOM.....	39
2.5.5	Manipolazione del DOM.....	39
2.6	Un confronto tra le librerie.....	40
2.7	Note finali.....	43
Capitolo 3:	Creazione prototipo.....	43
	Obiettivo.....	44
3.1	Scelta di un sito di riferimento.....	44
3.2	Analisi della struttura della pagina.....	45
3.3	Costruzione di uno scraper.....	47
3.3.1	Architettura dell'applicazione.....	49
3.3.2	Creazione del driver.....	50
3.3.3	Apertura della pagina.....	51
3.3.4	Gestione dei risultati.....	52
3.4	Una nota sulla frequenza delle richieste.....	53
3.5	Alcuni parametri di performance.....	53
3.5.1	Test locale di comparazione Selenium-Puppeteer.....	53
3.5.2	Una considerazione emersa in fase di test.....	56
3.5.3	Un'analisi con più driver.....	56
3.5.4	Una nota su Cheerio.....	58
Conclusioni.....		59

Ringraziamenti

Prima di entrare nel merito della trattazione, desidero esprimere la mia profonda gratitudine a tutte le persone che mi hanno sostenuto in questi ultimi tre anni. Questo periodo è stato per me di fondamentale crescita personale e intellettuale che mi ha permesso di acquisire una visione più matura e consapevole della realtà.

In primo luogo, vorrei ringraziare i miei genitori e mia sorella per il costante supporto morale, soprattutto nei momenti più complessi e sfidanti. Senza il loro incoraggiamento, affrontare queste difficoltà sarebbe stato molto più arduo.

Un sincero ringraziamento va anche ai miei nonni, che con le loro premure e attenzioni non hanno mai mancato di farmi sentire il loro affetto e la loro vicinanza.

Desidero rivolgere un ringraziamento speciale a mio zio, che fin dall'infanzia mi ha trasmesso l'importanza dello studio, un valore che ha profondamente segnato il mio percorso formativo.

Infine, un grazie al mio relatore, il Professor Bellavista, per la pazienza, i preziosi consigli e l'attenzione dedicata alla revisione e allo sviluppo di questo elaborato.

Premessa

Nel corso di questa tesi verrà sviluppato un prototipo software finalizzato all'estrazione e all'analisi di dati provenienti da fonti Web di natura finanziaria. È importante sottolineare che i risultati generati dal prototipo non devono essere interpretati come indicazioni affidabili o precise per scopi operativi o decisionali, in quanto l'obiettivo principale di questo lavoro è dimostrare le potenzialità tecniche del Web scraping e non fornire raccomandazioni finanziarie.

I dati e le analisi prodotti sono da considerarsi esclusivamente dimostrativi e non sono stati sottoposti a rigorosi controlli di accuratezza o validazione. Il prototipo sviluppato si basa su una selezione di fonti di dati pubblici e su tecniche standard di estrazione e rielaborazione, ma non tiene conto di variabili economiche o finanziarie che potrebbero influenzare i risultati in modo significativo.

Inoltre, la scelta di concentrarsi su dati finanziari riflette un interesse personale riguardante l'andamento delle fluttuazioni di mercato di alcuni strumenti finanziari, piuttosto che un intento professionale o scientifico di fornire previsioni o analisi di valore.

Introduzione

Nel panorama odierno del Web, l'enorme quantità di dati disponibili rappresenta una risorsa di inestimabile valore per l'analisi, l'elaborazione e la ricerca. In questo contesto, il Web scraping si è affermato come una delle tecniche più potenti per raccogliere in modo automatizzato dati accessibili pubblicamente su Internet. Attraverso l'uso di strumenti software e script appositi, il Web scraping consente di estrarre informazioni da pagine Web e di strutturarle in formati utilizzabili per molteplici scopi, dalla raccolta dati per analisi di mercato alla creazione di dataset per progetti di intelligenza artificiale e machine learning.

Questa tesi si concentra principalmente sugli aspetti tecnici del Web scraping, verranno analizzate le potenzialità offerte da diverse librerie e strumenti open-source. In particolare, verrà approfondito l'utilizzo di Selenium, una libreria che permette di interagire con pagine Web in modo simile a un utente umano, simulando un browser reale. Selenium si distingue per la sua capacità di affrontare pagine Web dinamiche, basate su tecnologie come JavaScript e AJAX, che caricano contenuti in modo asincrono, rendendo necessaria una gestione più complessa dell'estrazione dati rispetto a pagine statiche.

Il focus della tesi è quello di mettere in luce i vantaggi tecnici del Web scraping, evidenziando come, con strumenti relativamente semplici, sia possibile ottenere risultati significativi anche senza il ricorso a infrastrutture complesse. Attraverso l'implementazione di un piccolo applicativo, verrà dimostrato come Selenium possa essere utilizzato per gestire in modo efficiente l'estrazione di dati da pagine Web.

Una delle principali potenzialità del Web scraping risiede nella sua flessibilità. Grazie alla possibilità di personalizzare gli script in base alle esigenze specifiche, è possibile adattare il processo di estrazione a una vasta gamma di scenari. Che si tratti di estrarre dati da tabelle HTML, di scaricare immagini o di interagire con moduli e pulsanti, gli strumenti disponibili consentono di automatizzare gran parte del lavoro manuale che altrimenti sarebbe necessario per raccogliere tali informazioni.

Un altro tema trattato riguarda le strategie per migliorare l'efficienza del processo, riducendo al minimo il carico computazionale e massimizzando la quantità di dati raccolti in tempi ragionevolmente brevi. Verranno esaminati anche alcuni dei limiti tecnici legati all'uso di Selenium, come l'elevata occupazione di risorse hardware e la latenza introdotta dall'apertura di un browser, e saranno discusse soluzioni alternative, come l'uso di richieste HTTP dirette per il recupero di dati da pagine Web statiche.

In definitiva, la tesi si propone di offrire una visione completa delle potenzialità del Web scraping, mettendo in evidenza i suoi meccanismi tecnici e le soluzioni per sfruttare al meglio le capacità delle librerie.

Come è strutturata questa tesi

Capitolo 1

Nel primo capitolo viene fornita una panoramica generale del Web scraping, spiegandone il funzionamento e le diverse applicazioni. Vengono descritti i principali vantaggi, come la possibilità di automatizzare la raccolta di dati su larga scala, e alcuni svantaggi, come i problemi legati alla manutenibilità e ai sistemi di protezione dei siti Web.

Capitolo 2

Questo capitolo analizza in dettaglio la struttura e l'architettura di funzionamento delle principali librerie open source utilizzate per il Web scraping. Vengono mostrati i problemi più ricorrenti affrontati dai programmi di scraping, come il caricamento dinamico dei contenuti e i sistemi anti-bot. Inoltre vengono mostrate le diverse API offerte dalle librerie, mettendo in evidenza le loro funzionalità principali.

Capitolo 3

In questo capitolo viene descritta la progettazione e l'implementazione di un prototipo software basato sulla libreria Selenium descritta nel capitolo precedente. Vengono analizzate alcune strategie generali per l'implementazione dell'applicazione, mostrando i vari vantaggi e svantaggi delle diverse soluzioni proposte.

Infine viene effettuato un confronto di prestazioni tra Selenium e Puppeteer, mettendo in evidenza differenze in termini di velocità, efficienza e gestione delle risorse.

Capitolo 1: il potere dei dati

Nella società odierna la necessità di informazioni puntuali, precise e soprattutto tempestive rappresenta un elemento di primaria importanza.

Negli ultimi decenni, il ruolo dei dati nell'orientare le decisioni aziendali è diventato sempre più cruciale. L'era moderna è caratterizzata da un'incredibile quantità di informazioni disponibili, principalmente grazie alla diffusione di Internet e del "World Wide Web".

Questo enorme flusso di dati ha trasformato le informazioni stesse in uno dei beni più preziosi dell'economia contemporanea.

Settori come la finanza, il marketing, la sanità e molti altri traggono vantaggio dall'analisi dei dati per ottimizzare le loro operazioni e prendere decisioni strategiche più informate.

Tra le innumerevoli e fonti di informazione utilizzate quotidianamente dalle persone, spiccano i siti Web e i social media.

Alcune statistiche sono fornite da un articolo del Sole 24 Ore - "Informazione italiani, più social ma senza fidarsi troppo" il cui link è inserito nella bibliografia [1].

Più della metà stimata della popolazione fa uso massivo dei siti Web per ottenere informazioni, il che dimostra come il Web sia considerato uno dei luoghi virtuali più ricchi di informazioni.

Al giorno d'oggi alcune ricerche necessitano di grandi moli di dati, gran parte disponibili su piattaforme Web. La ricerca e l'analisi manuale richiede risorse significative in termini di tempo e capacità umane, e anche in questo caso, non è immune da errori.

1.1 Web scraping

Il Web scraping è un processo che consiste nell'uso di programmi software che hanno come scopo quello di estrarre informazioni presenti sui siti Web. Le risorse interessate spaziano dal semplice testo fino alle immagini e file audio

L'approccio generale prevede il collegamento ad un sito remoto per mezzo di un programma, il software provvederà a scaricare l'intero codice contenuto nella pagina Web o parte di esso rendendolo disponibile a delle operazioni che consentono di estrarne il contenuto.

A questo punto il programma, tramite alcuni comandi procede ad estrarre la parte di dati interessata e a memorizzarla in strutture dati complesse, rendendo così le informazioni strutturate e più facili da manipolare.

È essenziale analizzare la struttura del sito per individuare gli elementi di interesse, spesso identificati mediante tag HTML o selettori CSS i quali costituiranno il punto focale dell'estrazione dei dati.

Successivamente alla trasformazione dei dati in una forma strutturata, è consigliato salvare i risultati ottenuti in appositi supporti per la persistenza dei dati per non dover effettuare costantemente nuove richieste ai siti Web.

Le fasi successive riguardano l'analisi di dati, l'impiego di algoritmi anche di machine learning per cercare corrispondenze e formulare ipotesi sul lavoro svolto in precedenza.

Il processo può essere ripetuto quante volte è necessario per ottenere dati corretti e in tempi ristretti a seconda delle esigenze.

1.2 Vantaggi

Velocità

Un primo vantaggio della ricerca automatizzata è la velocità di reperimento dei dati rispetto alla ricerca manuale. Nell'arco di pochi secondi è possibile scaricare il contenuto di centinaia di siti Web contemporaneamente.

Scalabilità

Connesso al vantaggio della velocità, la scalabilità è data dalla capacità dei programmi di ottenere i dati necessari anche da innumerevoli siti Web contemporaneamente (superiori alle decine di migliaia).

1.3 Svantaggi

Manutenibilità

Uno svantaggio di tipo operativo riguarda la manutenibilità dei programmi. Essendo i software dipendenti dalla struttura delle risorse Web, anche un semplice cambiamento nel nome di un identificatore può bloccare l'intero processo.

È necessario che il programma sia scritto in modo da essere modificabile nella sola parte che riguarda l'identificazione delle risorse Web, lasciando l'intera struttura inalterata.

Rilevamento

Lo svantaggio principale dei programmi di Web scraping riguarda il loro rilevamento da parte di alcuni siti che non ne tollerano l'uso.

Ci sono vari modi in cui un sito Web può rilevare la presenza di richieste provenienti da agenti piuttosto che da utenti normali.

Il primo modo per cercare di rilevarne la presenza è l'analisi della frequenza di richieste effettuate da uno stesso endpoint, ovvero da uno stesso indirizzo IP.

Questa tecnica viene però elusa dall'utilizzo di programmi proxy i quali, cambiando ripetutamente sorgente IP (IP rotating), rendono l'analisi delle frequenze delle richieste una tecnica inadatta al rilevamento.

Un altro possibile tentativo che si può effettuare per evitare il blocco delle richieste è l'aggiunta di un'attesa variabile tra una richiesta e l'altra in modo da rendere la frequenza delle richieste più vicina a quella effettuata da un utente umano.

Un'altro metodo è il controllo della presenza di determinate firme nelle richieste HTTP tipiche di alcuni programmi che fanno uso di librerie note (le librerie saranno trattate nel prossimo capitolo).

Al momento del download, alcune di queste librerie contengono determinati valori di parametri presenti in alcune parti del loro codice sorgente.

Un modo per eludere questo controllo consiste nel ricompilare a mano il sorgente della libreria dopo aver modificato i parametri controllati dai siti Web.

1.4 Implicazioni Etiche e Legalità

È importante sottolineare che, nonostante i vantaggi offerti dal punto di vista della velocità di ricerca, questa pratica solleva anche questioni etiche e legali.

I programmi software possono effettuare innumerevoli richieste al secondo, agli occhi delle piattaforme Web visitate possono risultare un principio di attacco informatico.

Il reperimento di dati da siti Web può violare i termini di servizio delle piattaforme online, le leggi sulla privacy e la protezione dei dati in determinate giurisdizioni.

Inoltre, l'uso non autorizzato del Web scraping può danneggiare la reputazione di un'azienda e violare i diritti di proprietà intellettuale.

A questo scopo forniamo un link a recenti fatti di cronaca :

- “Google IA scraping as fair use” - bibliografia [3]
- “I bot rappresentano il 42% del traffico Web complessivo e i due terzi sono dannosi” - bibliografia [2]

È fondamentale quindi condurre le ricerche in modo etico e rispettoso delle normative vigenti, ottenendo il consenso appropriato o valutando attentamente i rischi e le implicazioni legali.

1.5 Scopo della tesi

Questa ricerca ha come obiettivo quello di far conoscere al lettore le tecniche di reperimento di dati presenti su siti Web, mostrare il processo che porta dal raggiungimento del sito fino alla creazione di strutture dati contenenti i dati scaricati.

Allo scopo verranno descritte le principali librerie software utilizzate odiernamente dettagliandone il funzionamento, alcuni dei principali limiti e una serie di confronti.

Verrà realizzato un prototipo di un programma software in grado di cercare informazioni relative ai prezzi dei titoli azionari quotati in borsa e alcuni dati relativi al bilancio d'esercizio. Sulla base dei dati ottenuti il programma software effettuerà un'analisi con lo scopo di fornire un giudizio sulla bontà del titolo.

Ci tengo specificare che la scelta del dominio del progetto, pur essendo in linea con l'argomento della tesi, non ha lo scopo di fornire un'analisi attendibile dei dati ottenuti.

La fase d'analisi è stata inserita per il solo scopo di mostrare la fase successiva all'ottenimento dei dati, ovvero quella di analisi.

1.6 Le librerie di web scraping

La maggior parte dei programmi utilizza alcune librerie per reperire dati via Web. Alcune di esse sono molto specifiche per un determinato browser, altre invece possono essere utilizzate anche su diversi motori di ricerca.

In questa tesi analizzeremo tre librerie, esse non sono le uniche presenti sul mercato, la scelta si basa sul fatto che godano di community abbastanza sviluppate e che possano essere utilizzate su più linguaggi di programmazione come Javascript e Python.

Le librerie che saranno analizzate nel capitolo successivo sono **Selenium**, **Cheerio**, **Puppeteer**.

Capitolo 2: architetture e librerie

Nel precedente capitolo abbiamo parlato degli scopi e finalità del Web scraping, ora vedremo nel dettaglio il funzionamento dei programmi.

La libreria Selenium sarà analizzata con maggior dettaglio in quanto verrà utilizzata per la realizzazione del prototipo di un'applicazione della quale si parlerà nel terzo capitolo.

2.1 Identificazione degli elementi

La ricerca degli elementi, indipendentemente dalla libreria, si basa principalmente su 2 dei seguenti metodi.

Selettori CSS

I selettori CSS consentono di specificare in modo preciso gli attributi degli elementi che si desidera scegliere. Ad esempio, è possibile selezionare tutti gli elementi di una determinata classe o tutti gli elementi figli di un elemento specifico.

Offrono una grande flessibilità e sono ampiamente supportati da molte librerie di Web scraping.

XPATH

XPath è un linguaggio di query utilizzato per navigare e selezionare elementi in un documento XML o HTML. È particolarmente utile quando si desidera identificarli in base alla loro posizione relativa all'interno della struttura della pagina Web. XPath consente di specificare percorsi gerarchici per individuare elementi specifici, il che lo rende particolarmente potente per l'identificazione di elementi complessi o annidati. Ad esempio, è possibile selezionare un elemento che si trova all'interno di un elemento padre di una classe specifica o che segue un determinato schema di struttura.

2.2 Un problema ricorrente

Ancor prima di iniziare l'analisi delle librerie è necessario considerare un problema tecnico molto frequente nelle applicazioni di scraping il quale, se non gestito a dovere, può far fallire il programma.

Sincronizzazione

Uno dei problemi principali che si incontrano nello sviluppo di un programma di Web scraping è la sincronizzazione tra programma e il browser Web controllato da quest'ultimo.

Non è possibile prevedere a priori quanto tempo sarà richiesto al sito per rispondere alla richiesta di visualizzazione di una pagina e il tempo impiegato dal browser per caricarla.

Un programma che vuole accedere ai dati scaricati dovrà assicurarsi siano disponibili nel momento in cui un determinato comando ne voglia fare uso.

Un esempio standard che evidenzia il problema in questione è la richiesta ad un generico sito che esegue codice Javascript sul browser locale il quale crea o modifica degli elementi.

Nel momento in cui la risposta giunge al browser, essa non contiene ancora gli elementi che il programma desidera, in quanto saranno pronti solo dopo l'esecuzione del codice lato browser.

In alcuni casi, è possibile non attendere l'intero caricamento della pagina se si vuole massimizzare la velocità di ricerca; questo però presenta il rischio che i dati non siano stati interamente caricati.

2.3 La libreria Selenium

Una delle librerie più utilizzate per l'automazione del Web scraping è Selenium. In questa sezione esamineremo l'architettura e il funzionamento, concentrandoci sulla sua versione più recente, Selenium 4.

Selenium è una libreria utilizzata per molteplici finalità, principalmente per automazione Web, test automatici di siti e Web scraping.

Automatizzare il browser

La funzione principale che svolge la libreria è quella di automatizzare un Web browser per la ricerca di informazioni remote.

Permette di simulare comuni attività svolte da utenti normali, ad esempio il click di un bottone, l'inserimento di parole in un campo di testo, la navigazione tra diverse risorse e lo scaricamento di documenti.

Cross browser

Un altro principio guida di Selenium è il supporto a diversi browser.

Il codice delle funzioni Selenium non richiede modifiche quando si sceglie di cambiare browser, a parte ovviamente il cambiamento del nome del browser.

Componenti principali della libreria

L'architettura di base di Selenium si basa sui seguenti componenti : il **browser**, il **driver** e il **WebDriver**.

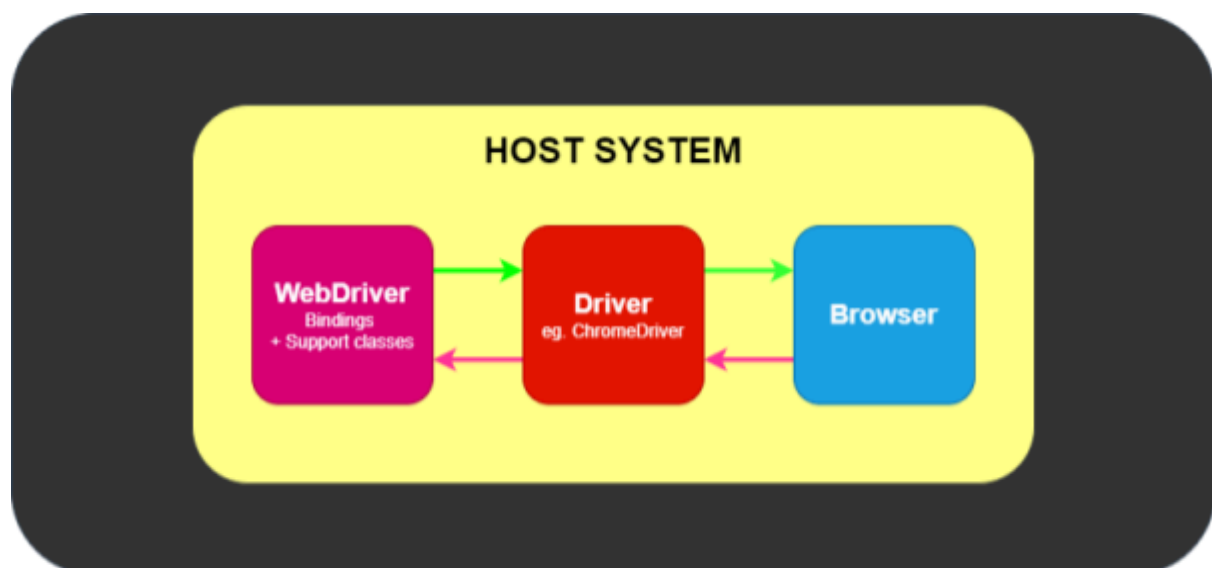


Figura 2.1 - Architettura della libreria Selenium -
https://www.selenium.dev/images/documentation/webdriver/basic_comms.png

- **Browser (Web browser)** : applicazione che permette di accedere a siti Web mediante l'utilizzo del protocollo HTTP.
Selenium offre la scelta di utilizzare come browser Chrome, Firefox, Edge, Safari ed Explorer.

- **Driver** : controlla il comportamento del browser. Può essere scaricato gratuitamente a seconda del motore di ricerca che si vuole utilizzare sui siti dei provider ufficiali. Selenium infatti è compatibile con vari browser, rendendolo molto efficace e portabile.
Riceve istruzioni da WebDriver e le inoltra al browser, può essere pensato come realizzazione specifica del WebDriver a seconda del browser.
- **WebDriver** : il ruolo più importante dell'intera architettura di Selenium è svolto da questo componente ovvero un'interfaccia che fa uso di API e protocolli per controllare indirettamente il Web browser attraverso il driver.
Vengono implementati dai Driver a seconda del motore di ricerca scelto.

2.3.1 Installazione di Selenium

Uno dei principali vantaggi di Selenium è la sua vasta gamma di scelta tra vari linguaggi di programmazione su cui implementare l'applicazione.

Le possibili scelte sono Python, Ruby, C# , Javascript , PHP, Java.

In questa sede forniremo degli esempi su Python e Javascript che sono al momento le scelte più frequenti nella comunità degli sviluppatori di applicazioni Web scraping.

L'utilizzo della libreria Selenium richiede prima di tutto l'installazione del WebDriver Selenium. I comandi rispettivamente Python e Javascript sono i seguenti

- ★ `python -m pip install -U selenium`
- ★ `npm install selenium-webdriver`

I comandi sopra citati permettono di scaricare il componente principale della libreria. Come vedremo nei prossimi paragrafi esso è l'unico componente che necessita di installazione manuale a seguito dell'introduzione del modulo DriverManager nelle versioni più recenti di Selenium 4.6

2.3.1.1 Installazione del driver

Il WebDriver impartisce indicazione al driver il quale a sua volta controlla le operazioni del browser. Essendo Selenium cross-platform, occorre installare un driver diverso in base alla scelta effettuata.

Selenium fornisce indicazioni dei siti ufficiali dove è possibile andare a reperire la versione del driver più opportuna.

I driver sono scaricabili gratuitamente dai siti ufficiali, ma tutto ciò non è necessario, come si potrà evincere nel paragrafo seguente.

I browser supportati sono indicati di seguito:

Chrome	https://chromedriver.chromium.org/downloads
Edge	https://developer.microsoft.com/it-it/microsoft-edge/tools/webdriver/?form=MA13LH
Firefox	https://firefox-source-docs.mozilla.org/testing/geckodriver/index.html

2.3.1.2 Selenium Driver Manager

Come indicato precedentemente, è necessario inserire un driver tra il WebDriver e il browser che il programma controllerà.

Le prime versioni di Selenium prevedevano lo scaricamento manuale di un driver specifico (Chrome Driver, Gecko Driver, Ms Edge Driver) e il collegamento del percorso assoluto del driver al programma inserendolo nelle sue variabili d'ambiente.

La gestione delle versioni della libreria doveva essere quindi gestita in modo manuale. Una volta scaricato il driver per una certa versione del proprio browser locale, esso rimaneva tale per sempre o almeno finché lo sviluppatore manualmente non decideva di cambiarlo.

Il problema è che il browser viene rinnovato automaticamente a fronte di aggiornamenti, quindi la gestione manuale delle corrette versioni della libreria può risultare difficoltosa.

Nelle più recenti versioni di Selenium, in ogni linguaggio di programmazione, è stato aggiunto un modulo denominato Selenium Driver Manager.

Il modulo è già presente al momento dello scaricamento del webdriver e viene utilizzato solo nel momento in cui il driver non è presente localmente.

Lo sviluppatore è libero di continuare a gestire i driver manualmente oppure può affidarsi ad un gestore esterno, in questo caso proprio il Selenium Driver Manager.

All'esecuzione di un comando Selenium, se il driver non è fornito esplicitamente da parte dello sviluppatore, il compito di gestire i driver sarà affidato al Selenium Driver Manager.

La gestione dei driver Selenium per mano del Driver Manager si compone delle seguenti fasi:

1. **Verifica della versione del browser (browser discovery)**: Selenium scopre che versione del browser si sta utilizzando
2. **Verifica della versione del driver**: dopo aver rilevato la versione del browser, Selenium provvede a scaricare la corretta versione del driver dai siti ufficiali a seconda di quale browser si è scelto.
3. **Scaricamento del driver**: il driver viene scaricato localmente nella cache
4. **Cache**: per evitare che il processo venga ripetuto per ogni comando imposto ad driver, esso viene salvato nella cache.

2.3.1.3 Selenium Browser Manager

Come già introdotto nei paragrafi precedenti, Selenium offre la possibilità di scegliere quale browser utilizzare. Non è quindi necessario aver installato uno specifico browser per il corretto funzionamento del programma, poiché, oltre alla gestione dei driver, Selenium gestisce anche le versioni dei browser tramite il Selenium Browser Manager.

1. **Verifica se il browser è installato**
 - a. Se il browser non è installato, a default, Selenium scarica la versione stabile supportata.
 - b. È possibile impostare una versione diversa, più vecchia o più recente in base alle necessità.
2. **Download della versione del browser**
3. **Caching della versione del browser**

Dal momento che il browser viene aperto viene creata una sessione.

Le sessioni vengono gestite in modo diverso in base al tipo di driver che può essere locale o remoto.

Il driver locale viene eseguito sulla macchina locale e la sua sessione viene gestita dalla classe Service Class Driver che permette di specificare il path del driver e su che porta collegare il driver.

I dati relativi al driver e al browser vengono salvati nella home directory dell'utente.

Il Selenium Manager utilizza un file `see-metadata.json` in cui salva tutte le versioni dei browser per un certo tempo, denominato TTL, dopo il quale effettua una nuova richiesta per trovare la versione aggiornata.

ESEMPI

JavaScript

```
//Importiamo i moduli necessari di Selenium
const {Builder} = require('selenium-webdriver');

//Definiamo il driver per il browser
let driver = new Builder().forBrowser('chrome').build();

//Se volessimo utilizzare un driver pre-scaricato possiamo
utilizzare //il seguente codice
let chromedriver = require('chromedriver').path;
//let driver = new
Builder().forBrowser('chrome').setChromeOptions(new
chrome.Options().setChromeBinaryPath(chromedriver)).build(
);
```

Python

```
#Importiamo i moduli necessari di Selenium
from selenium import webdriver

#Il primo passo è definire il driver del browser, in questo
esempio utilizzeremo Chrome
#Nella seguente istruzione non inseriamo un alcun percorso.
Questo perché Selenium tramite il driver manager scaricherà
automaticamente il driver di Chrome
driver = webdriver.Chrome()

#Un'alternativa è quella di specificare il percorso del driver
di Chrome
#driver =
webdriver.Chrome(executable_path="C:/path/to/chromedriver.exe"
)

#Essendo il browser di Chrome già installato, non è necessario
l'intervento del Selenium browser manager.
```

```
#Se il browser non è installato, Selenium browser manager lo scaricherà automaticamente.
```

2.3.2 Flusso di lavoro di Selenium

Una volta installato Selenium e scaricato il relativo driver manualmente o per mezzo del Selenium Driver Manager, si può iniziare ad operare normalmente.

Descriveremo ora il funzionamento alla base della libreria.

Ricordiamo che i componenti principali sono il webdriver, il driver e il browser

Il flusso di lavoro di Selenium inizia con la richiesta di esecuzione di un comando da parte del client Selenium, che è il programma che esegue dei comandi della libreria Selenium.

Le istruzioni possono variare dalle azioni più semplici, come il clic su un elemento, al recupero di dati più complessi, come l'accesso a un elemento specifico della pagina Web.

Un esempio di comando può essere la localizzazione di un elemento specificando il suo identificativo.

```
const loc = await driver.findElement(By.ID('information'));
```

La richiesta di esecuzione viene inoltrata al WebDriver, che agisce come un intermediario tra il client Selenium e il browser.

Selenium è una libreria compatibile con il W3C ovvero che implementa le linee guida del World Wide Web Consortium. Il consorzio stabilisce che ogni comando deve essere codificato per mezzo di una singola richiesta HTTP e viceversa ogni singola richiesta HTTP rappresenta un singolo comando.

Riprendendo l'esempio precedente la richiesta HTTP inviata al webdriver è la seguente : POST /session/{sessionId}/element/{elementId}/element

in cui sessionId è l'id dell'istanza del driver, elementId è l'ID che si vuole cercare.

Per una più approfondita mappatura tra comandi e richieste HTTP si faccia riferimento al sito ufficiale della W3C inserito nella bibliografia.

Il Webdriver deserializza la richiesta ricevuta e la processa, traducendo il comando in istruzioni comprensibili per il browser.

Il browser esegue le azioni richieste e invia una risposta al Webdriver che a sua volta le invia al client Selenium.

Questo ciclo si ripete fino alla fine del programma Selenium.

Osserviamo un breve esempio per mettere in evidenza le varie fasi.

```
//1. importare i moduli necessari di Selenium
const {Builder} = require('selenium-webdriver');

//2. definire il driver per il browser
let driver = new Builder().forBrowser('chrome').build();

//3. collegarsi al sito di nostro interesse
driver.get('https://www.google.com');

// ... Accesso agli elementi della pagina web

//4. chiudere il driver
driver.quit();
```

Figura 2.2 - Fasi di un programma Selenium

- la prima è relativa all'installazione di tutte le dipendenze necessarie per il corretto funzionamento della libreria.
- successivamente viene creato un driver per il browser chrome. Notiamo che questa istruzione richiede l'intervento del Selenium Driver Manager, questo perchè nell'esempio non è stato specificato un percorso esplicito per un driver. Selenium si occuperà di andare a scaricare il driver più opportuno. Successivamente, il Selenium Browser Manager si occuperà di reperire una versione del browser compatibile con il driver appena scaricato.
- conclusa la fase di preparazione, ovvero nel momento in cui il driver viene creato, è possibile effettuare il collegamento ad una risorsa Web tramite il metodo `get()`, specificando l'indirizzo url del sito interessato.
- questo comando verrà recapitato al Selenium WebDriver attraverso una richiesta HTTP di tipo POST con all'interno specificato l'url del sito e l'identificativo del driver che ha effettuato la richiesta.

Analogamente tutto ciò avviene anche per il comando `quit`, ciò che cambiano sono gli header della richiesta HTTP inviata al webdriver.

Analizzeremo ora come viene caricata la pagina Web dal driver Selenium e come i problemi di sincronizzazione a cui accennavamo ad inizio capitolo possano essere risolti.

2.3.3 Strategie di caricamento della risorsa

Per poter accedere ai diversi elementi della pagina Web occorre che essi siano stati scaricati dall'endpoint e mostrati sul browser locale, in caso contrario risulteranno inaccessibili.

Selenium implementa 3 diverse strategie di scaricamento delle pagine Web. Esse valutano lo stato del parametro `readystatechange` della pagina Web che indica lo stato di scaricamento della stessa.

Una prima soluzione può essere quella di attendere per un certo limite di tempo prima di ogni richiesta ad esempio 5 secondi. Il problema che emerge è che il codice non può sapere quanto tempo impiegherà la libreria per rendere disponibile il dato, un'attesa troppo breve può provocare un fallimento del programma mentre una troppo lunga andrebbe incontro a problemi di inefficienza.

Strategia di default

Si attende che tutte le risorse siano state interamente scaricate. Utile nel caso si voglia essere sicuri che l'intero contenuto sia presente in memoria.

Strategia eager

È possibile accedere agli elementi del DOM della pagina, ma risorse come le immagini potrebbero non essere state scaricate interamente.

È una buona scelta nel caso si sia interessati esclusivamente ad informazioni testuali.

Strategia senza attesa

Il webdriver non si blocca e l'esecuzione del codice procede senza attesa.

Molto veloce e allo stesso tempo molto rischiosa dato che spesso le istruzioni successive alla richiesta della pagina Web tentano di accedere agli elementi della pagina scaricata.

Se essi non sono presenti tutte le funzioni successive di accesso alle informazioni falliranno.

Spesso molte pagine sono modificate dinamicamente data la presenza di codice javascript, perciò il fatto che una pagina sia stata completamente scaricata non significa che tutti gli elementi siano disponibili.

Una prima soluzione è aggiungere un comando di attesa di svariati secondi dopo l'istruzione di connessione alla pagina per avere la certezza del caricamento di tutti gli elementi interessati . Se l'attesa è troppo alta, il tempo di reperimento dati può aumentare considerevolmente. Selenium prevede 2 meccanismi per risolvere questo problema.

Attesa implicita

Si imposta il driver con un tempo massimo di attesa globale identico per ogni elemento durante tutta la sessione.

Il valore di default impostato è 0, se l'elemento non viene trovato viene restituito un errore.

Nel momento in cui l'elemento risulterà disponibile, anche prima del limite massimo, sarà estratto e l'esecuzione procederà.

Per impostare un'attesa implicita occorre specificarlo tra le opzioni del driver

```
await driver.manage().setTimeouts({ implicit: 2000 });
driver.implicitly_wait(2)
```

Attesa esplicita

Utile per specificare condizioni di attesa specifici per diversi elementi della pagina.

Viene imposto un limite di tempo locale che può essere diverso per ciascun elemento, dopo il quale l'istruzione ritorna un timeout.

Ogni 500ms il webdriver verificherà a polling la condizione specificata finchè non sarà verificata oppure fino allo scadere del tempo massimo.

```
await
driver.get('https://www.selenium.dev/selenium/web/dynamic.html');
let revealed = await driver.findElement(By.id("revealed"));
await driver.findElement(By.id("reveal")).click();
await driver.wait(until.elementIsVisible(revealed), 2000);
await revealed.sendKeys("Displayed");
```

```
wait = WebDriverWait(driver, timeout=2)
wait.until(lambda d : revealed.is_displayed())
```

È possibile specificare l'intervallo di polling, quali eccezioni debbano essere automaticamente gestite, il timeout totale.

```
errors = [NoSuchElementException,
ElementNotInteractableException]
wait = WebDriverWait(driver, timeout=2, poll_frequency=.2,
ignored_exceptions=errors)
wait.until(lambda d : revealed.send_keys("Displayed") or
True)
```

2.3.4 Accesso agli elementi

Dopo il caricamento dei dati sul browser è necessario accedervi per estrarre le risorse di interesse.

La parte principale di ogni programma Selenium è l'interazione con gli elementi della pagina a cui ci si è collegati precedentemente.

Localizzatori di elementi

Selenium supporta una grande varietà di localizzatori.

- **Classe** → `driver.find_element(By.CLASS_NAME, "information")`
- **Selettore CSS** → `driver.find_element(By.CSS_SELECTOR, "#fname")`
- **ID** → `driver.find_element(By.ID, "lname")`
- **Nome** → `driver.find_element(By.NAME, "name")`
- **tag** → `driver.find_element(By.TAG_NAME, "tag")`

Localizzatori relativi

A volte può essere difficile impostare o costruire un localizzatore per accedere ad un determinato elemento direttamente.

Un metodo ulteriore per trovare una risorsa sulla pagina è in base alla sua posizione.

Se un elemento della pagina Web vicino a quello al quale siamo interessati ha un localizzatore semplice da definire allora si possono utilizzare vari metodi per provare ad accedervi.

Above

```
email_locator = locate_with(By.TAG_NAME, "input").above({By.ID:
"password"})
```

Below

```
password_locator = locate_with(By.TAG_NAME, "input").below({By.ID: "email"})
```

Left Of

```
cancel_locator=locate_with(By.TAG_NAME, "button").to_left_of({By.ID: "submit"})
```

Right Of

```
submit_locator = locate_with(By.TAG_NAME, "button").to_right_of({By.ID: "cancel"})
```

Near

```
email_locator = locate_with(By.TAG_NAME, "input").near({By.ID: "lbl-email"})
```

Un esempio di un semplice script

```
#Questo script Python apre un browser, si connette a Google, cerca "web
scraping" e stampa i primi 5 risultati.
#Per eseguire questo script, è necessario installare il modulo
Selenium.
#Per installare il modulo Selenium, aprire il terminale e digitare il
seguente comando: pip install selenium
#Inoltre, è necessario scaricare il driver del browser. In questo caso,
stiamo utilizzando il driver di Chrome.

#Importiamo i moduli necessari
from selenium import webdriver
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.common.by import By
import time

#Primo passo: definire il driver del browser, in questo caso stiamo
utilizzando il driver di Chrome ovvero chromedriver
print("Apertura del browser...")
driver = webdriver.Chrome()
```

```

#Secondo passo: connettersi a Google, oppure a qualsiasi altro sito web
print("Connessione a Google...")
driver.get("https://www.google.com")

#-----
#Passo aggiuntivo: rimuovere il pop-up che chiede di accettare i cookie
#Rimozione del pop up che chiede di accettare i cookie
time.sleep(40)
driver.implicitly_wait(3)
#cookie = driver.find_element(By.ID, "L2AGLb") #ID del bottone
"Accetta"
cookie = driver.find_element(By.ID, "W0wltc") #ID del bottone "Rifiuta"
cookie.click()
#-----

#Terzo passo : identificare l'elemento di ricerca
print("Ricerca di 'web scraping'...")
driver.implicitly_wait(3)
search_box = driver.find_element(By.NAME, "q") #q è il nome
dell'elemento di ricerca su Google

#Attendi 3 secondi
time.sleep(3)
search_box.send_keys("web scraping")
#Premi il tasto invio
time.sleep(3)
search_box.send_keys(Keys.ENTER)
#Attendi 3 secondi
time.sleep(3)
print("Stampa dei primi 5 risultati...")
results = driver.find_elements(By.CLASS_NAME, "g")

print() #Andiamo a capo
for i in range(5):
    print("Risultato", i+1)
    print(results[i].text)
    #Andiamo a capo
    print()

print("Chiusura del browser...")
driver.quit()

```

```
print("Fine dello script.")
```

2.3.5 Selenium Grid

Fin ora abbiamo analizzato il funzionamento locale di Selenium, ovvero quando le istruzioni e il browser risiedono sulla stessa macchina.

Il Selenium Grid è un server che permette l'esecuzione parallela di script Selenium su diverse macchine remote.

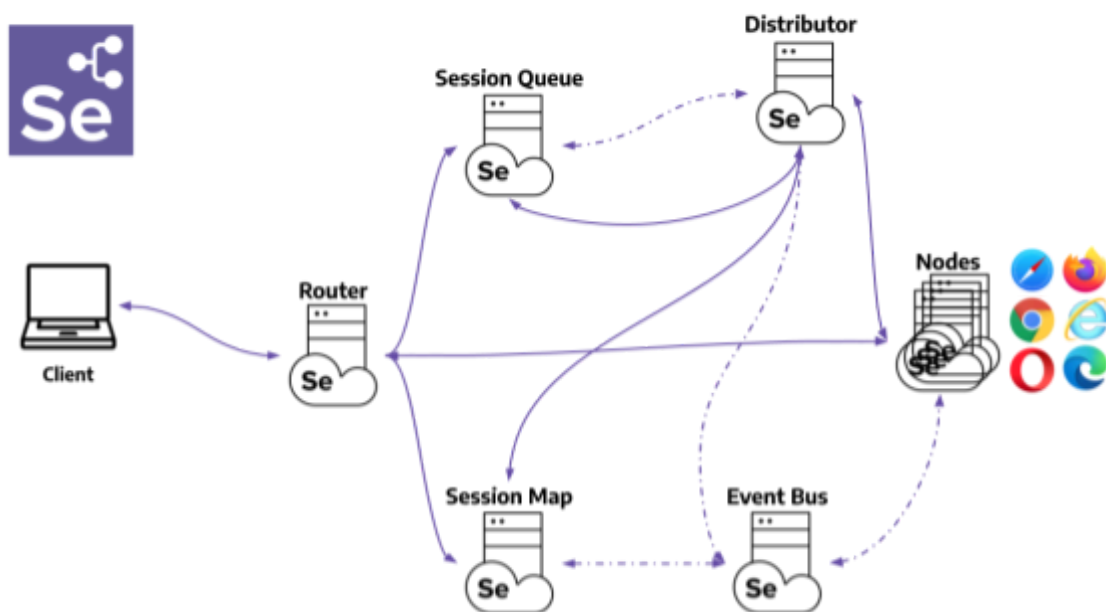


Figura 2.3 - Architettura Selenium Grid -
<https://www.selenium.dev/images/documentation/grid/components.png>

1. Router

Componente entry point che riceve le richieste dal client e le esegue sui vari nodi al quale è collegato.

Se la richiesta specifica di aprire una nuova sessione, quindi un nuovo browser, verrà inoltrata al componente New Session Queue

Se la richiesta si riferisce ad una sessione già esistente, il router interrogherà il componente Session map per ottenere l'id del nodo sul quale la sessione è in esecuzione.

Successivamente la richiesta verrà inoltrata al nodo.

2. Distributor

Tiene traccia di tutte le capacità dei vari nodi sulla rete. Consulta periodicamente la New Session Queue verificando l'esistenza di nuove richieste in attesa.

Cerca così un nodo ideale sul quale poter aprire la nuova sessione.

Dopo aver smistato la richiesta al nodo, rimuovendola dalla coda, inserisce una corrispondenza nel Session Map.

Rimane in ascolto di richieste di connessione da parte dei nodi

3. Session map

Componente nel quale si tiene traccia delle corrispondenze nodo-sessione. Viene periodicamente interrogato dal router all'arrivo di una richiesta e aggiornato dal Distributor

4. New Session Queue

Coda di richiesta gestita con una politica di tipo FIFO. È possibile configurarla in modo da limitare il tempo di accodamento di ogni richiesta.

Il router aggiunge nuove richieste in coda, periodicamente il new session queue verifica che le richieste non siano scadute, in tal caso vengono rimosse dalla coda.

Il Distributor estrae nuove richieste dalla coda e cerca di smistarla in un nodo che soddisfi le condizioni della richiesta.

In caso di fallimento il Distributor raggiungerà la richiesta alla coda

5. Nodo

Si registra presso il Distributor attraverso il Bus inviando una richiesta con le sue capacità non appena viene avviato.

A default il nodo registra tutti i browser disponibili sulle sue variabili di ambiente, creando uno slot per ogni CPU che possiede.

Ogni nodo esegue i comandi ricevuti senza preoccuparsi di effettuare alcun controllo.

All'avvio e anche dopo la registrazione il nodo invia periodicamente un oggetto JSON al Distributor contenente vari campi:

- stato
 - UP : disponibile a ricevere nuove richieste
 - DRAINING : il nodo sta elaborando la richiesta, quindi nessuna nuova richiesta deve essere inviata a questo nodo
 - DOWN : il nodo non è disponibile
- slots : array contenente informazione relative ai vari slot che la macchina può ospitare
 - ID : identifica l'id dello slot
 - capacità : informazioni relative allo slot, come ad esempio quali browser può eseguire.
 - session : identifica la sessione che si sta eseguendo
- versione : corrisponde alla versione di Selenium che il nodo utilizza

Event bus

Componente utilizzato per scambiare messaggi tra gli altri componenti. I messaggi vengono inviati su questo canale.

I messaggi che viaggiano su questo bus sono di tipo asincrono.

2.3.6 BiDirectional API

Selenium si basa sul modello di funzionamento di un WebDriver, che implementa un protocollo di tipo richiesta-risposta. Attualmente, Selenium e le varie aziende proprietarie di browser stanno lavorando ad un'evoluzione del protocollo : webdriver bidirectional protocol.

L'obiettivo è aggiungere al protocollo attuale la capacità di essere bidirezionale attraverso la tecnologia websockets.

Il protocollo renderà più facile l'utilizzo della libreria eliminando i problemi di attesa implicita o esplicita necessari per garantire la sincronizzazione tra programma Selenium e browser.

L'evoluzione del protocollo è visitabile al seguente link
<https://w3c.github.io/webdriver-bidi>

2.4 La libreria Puppeteer

Puppeteer è una libreria di Node.js che offre una vasta gamma di API di alto livello per controllare il flusso di lavoro del browser Chrome.

- offre funzionalità molto simili a Selenium tra cui l'automazione di browser, testing di siti web, test di varie estensioni di browser, screenshot e pdf di pagine.
- è una libreria molto più specifica rispetto a Selenium, supporta solo i browser Chrome e Mozilla.

Gli obiettivi principali di questa libreria sono:

- ★ semplicità
- ★ velocità
- ★ stabilità
- ★ sicurezza

2.4.1 Installazione

Per installare Puppeteer è sufficiente eseguire il seguente comando

```
npm i puppeteer
```

Tutto viene installato in \$HOME_DIR_USER/.cache/puppeteer.

Puppeteer non necessita di alcuna configurazione, al momento dell'installazione è già pronto per l'uso.

Al momento dell'installazione viene scaricata una versione specifica di Chrome, in caso se ne voglia utilizzare una specifica occorre specificarlo al momento del lancio del browser

```
const browser = await puppeteer.launch({executablePath: '/path/to/Chrome'});
```

2.4.2 Modalità headless

A default Puppeteer non apre visivamente un browser come nel caso di Selenium. Si dice che lavora in modalità headless, questo è utile in caso si utilizzino più script in parallelo per ottenere dati da diverse pagine.

In caso si vogliano vedere le operazioni effettuate dal browser è possibile specificarlo con il parametro `headless` nel comando `launch`.

```
const browser = await puppeteer.launch({headless: false});
```

2.4.3 Gestione del browser

Puppeteer offre una serie di API intuitive e di facile uso con le quali è possibile realizzare semplici script.

Le funzionalità principali offerte, comuni a tutte le altre librerie di scraping e test , sono l'apertura di un browser, la connessione ad una risorsa web e la manipolazione del DOM tramite altre API offerte.

2.4.4 Architettura

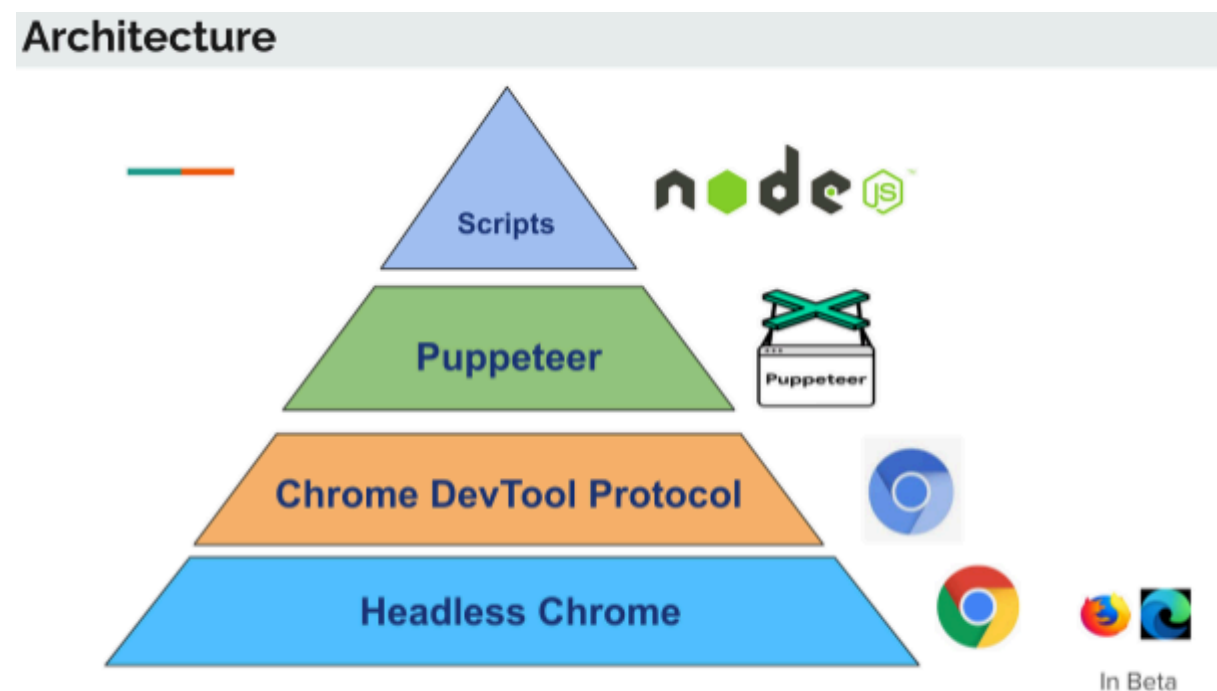


Figura 2.4 - Architettura Puppeteer -
https://miro.medium.com/v2/resize:fit:828/format:webp/1*uMaJYIX48nKlf7LDusoQdg.png

L'architettura di Puppeteer si compone di vari livelli:

- **Browser Chrome o Mozilla** : similmente a Selenium, alla base si trova un browser controllato dalla libreria Puppeteer.
- **Chrome DevTool Protocol** : un set di strumenti integrato nel browser Google Chrome che consente agli sviluppatori di ispezionare, modificare e debuggare il codice delle pagine web in tempo reale.
- **Puppeteer** : in questo caso facciamo riferimento a tutti i comandi messi a disposizione della libreria per interagire con il browser.
- **Script** : il programma che esegue comandi della libreria

2.4.5 Lancio del browser

```
import puppeteer from 'puppeteer';
const browser = await puppeteer.launch();
#Puppeteer utilizzerà il browser di default dato che il metodo
.launch è stato invocato senza parametri
const page = await browser.newPage();
```

2.4.6 Chiusura del browser

```
await browser.close();
```

2.4.7 Connessione ad un browser già aperto

È possibile connettersi direttamente ad un browser già aperto specificando la websocket alla quale il browser è connesso

```
const browser = await puppeteer.connect({
  browserWSEndpoint: 'ws://127.0.0.1:9222/...',
});
const page = await browser.newPage();
await page.goto('url');
```

```
#Interazione con gli elementi della pagina
```

```
browser.disconnect();
```

La differenza tra il metodo `close` e `disconnect` è che quest'ultimo non chiude il browser bensì il programma si disconnette dal browser permettendo all'utente di interagire.

2.4.8 Un template classico

Il tutto può essere riassunto dal seguente script, il quale mostra la semplicità con la quale è possibile reperire risorse via web.

```
import puppeteer from 'puppeteer';
const browser = await puppeteer.launch();
const context = await browser.createBrowserContext();

const page1 = await context.newPage();
const page2 = await context.newPage();

await page1.goto('url');
#Interazione con gli elementi della pagina

await page1.close();
await page2.close();

await context.close();
```

2.4.9 Interazione con gli elementi della pagina

L'iterazione con gli elementi della pagina avviene tramite i localizzatori (`locators`), il cui obiettivo, come per Selenium, è localizzare elementi nella pagina web specificando determinati criteri. Sebbene logicamente siano simili, ovvero adempiono al medesimo scopo, presentano importanti differenze a livello di funzionamento.

Il vantaggio dei localizzatori di Puppeteer rispetto a quelli di Selenium è che essi garantiscono alcune postcondizioni sull'esistenza dell'elemento. Aspettano che l'elemento specificato sia presente all'interno della pagina web, ovvero che sia visibile oppure nascosto.

In altri termini, utilizzando Puppeteer si evitano tutti i problemi di sincronizzazione presenti in Selenium.

Invocando metodi come click oppure l'invio di testo all'interno di elementi come input text oppure textarea, il locator aspetterà che essi siano abilitati.

1. Cliccare su un elemento

```
await page.locator('IDbutton').click();
```

2. Impostare criteri per la ricerca di un elemento

Il metodo filter permette di specificare alcuni criteri per la ricerca di un elemento

```
await page.locator('button').filter(button => !button.disabled)
.click();
```

3. Accesso in scrittura a campi di testo

```
await page.locator('input').fill('value');
```

4. Impostare eventi su un elemento

```
let willClick = false;
await page.locator('button')
.on(LocatorEvent.Action, () => {
  willClick = true;
})
.click();
```

2.4.10 Velocità di Puppeteer

La libreria Puppeteer è conosciuta principalmente per la sua velocità di esecuzione. I comandi impartiti dal programma javascript vengono trasmessi da Puppeteer a Chrome tramite il protocollo Chrome Developer Tool Protocol (CDP) il quale impartisce comandi al browser.

Ogni comando o evento è serializzato mediante un oggetto JSON.

Supportando esclusivamente i browser Chrome e Firefox, essendo scritto interamente in javascript, non necessita di scaricare componenti come i driver riducendo in questo modo il numero di layer da 3 (webdriver-driver-browser) a 2

(Puppeteer - browser).

2.5 La libreria Cheerio

L'ultima libreria che analizzeremo è Cheerio, libreria di Node.js basata su JQuery il cui uso è specifico per il Web scraping.

Cheerio è un parser di linguaggi di markup, offre diverse API per analizzare il contenuto di pagine contenenti codice HTML.

2.5.1 Architettura

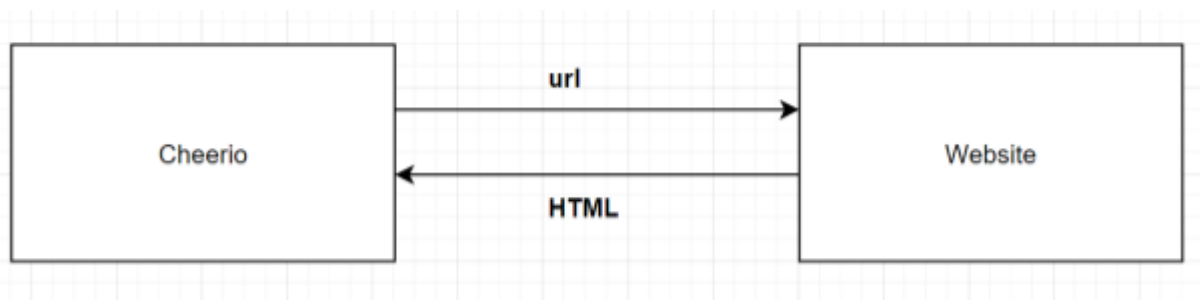


Figura 2.5 - Architettura Cheerio

L'architettura della libreria Cheerio è molto più semplice rispetto alle librerie prima descritte.

Da una parte abbiamo la **libreria Cheerio** che effettua una richiesta http ad un endpoint specifico.

Il sito Web risponde con del codice HTML.

La differenza fondamentale a differenza delle altre librerie è che **Cheerio non fa uso di un web browser**. Non è possibile navigare tra le pagine utilizzando i bottoni, che invece potevano essere controllati dai browser di Selenium e Puppeteer.

Le uniche azioni possibili sono quelle di analisi dell'HTML ricevuto, quindi la classica localizzazione e ricerca di elementi specifici.

Controllo di sintassi

È possibile validare il codice HTML ottenuto dalla pagina web oppure costruito direttamente a mano per mezzo di alcuni metodi forniti.

Velocità

La funzione principale di Cheerio è il parsing di HTML, è pensato per essere utilizzato una volta che si è ottenuto l'HTML della pagina.

Cheerio non è un web browser, non fornisce un'interfaccia visuale, non apre browser, non produce alcun output.

Questo rende tutto il processo di reperimento degli elementi estremamente veloce ed efficiente.

Flessibilità

Cheerio è in grado di analizzare sia codice HTML sia codice XML.

2.5.2 Il caricamento degli elementi

In generale il meccanismo di caricamento di documenti prende in ingresso codice HTML e restituisce un oggetto Cheerio con il quale è possibile interagire per analizzarne la struttura interna.

Per caricare i documenti sottoforma di codice HTML, Cheerio mette a disposizione vari metodi.

Load

Il primo modo, quello più semplice, per caricare documenti HTML o XML è attraverso il metodo `load`.

```
import * as cheerio from 'cheerio';
const $ = cheerio.load(
  '<html><head><title>Hello, world!</title></head></html>',
);
```

```
console.log($('h1').text());
// Output: Hello, world!
```

load Buffer

Simile al load precedente, ma dà la possibilità di caricare il documento direttamente da un file invece che inserire direttamente una stringa.

```
import * as cheerio from 'cheerio';
import * as fs from 'fs';

const buffer = fs.readFileSync('document.html');

const $ = cheerio.loadBuffer(buffer);

console.log($('title').text());
// Output: Hello, world!
```

from URL

Caricamento del documento HTML direttamente da un url . Il metodo è asincrono, è quindi necessario aggiungere await prima di invocare il comando per evitare tutti i problemi relativi alla sincronizzazione affrontati precedentemente.

```
const $ = await cheerio.fromURL('https://example.com');
```

2.5.3 Selezione degli elementi

Cheerio permette di selezionare gli elementi dell'oggetto Cheerio ottenuto come risultato del caricamento del documento attraverso dei selettori CSS.

Essendo Cheerio basato su JQuery, la selezione degli elementi è analoga. Nonostante non sia obbligatorio, è suggerito inserire il prefisso \$ prima del nome della variabile per non confonderlo con elementi di un browser.

- ☐ Selezione di tutti i paragrafi testuali

```
const $p = $('p');
```

- ☐ Selezione di tutti i tag di una determinata classe

```
const $myClass = $('.myClass');
```

- ☐ Selezione di tutti i tag con un certo valore di un certo attributo

```
const $selected = $('[data-selected=true]');
```

- ☐ Selezione di tutti i tag di un certo tipo, di una determinata classe

```
const $selected = $('p.selected');
```

2.5.4 Attraversamento del DOM

- *find*

Localizza un determinato elemento a partire dal suo selettore CSS.

```
const listItems = $('ul').find('li');
```

- *children*

Fornisce un array di tutti i figli diretti di un elemento specificato.

```
const listItems = $('ul').children('li');
```

- *parent*

Seleziona il genitore diretto di un elemento dato come input

```
const list = $('li').parent();
```

- *closest*

Seleziona l'antenato più vicino che soddisfa un selettore CSS dato come input

```
const list = $('li').closest('ul');
```

2.5.5 Manipolazione del DOM

Modifica di proprietà e attributi degli elementi

```
$('#img').attr('src', 'https://example.com/image.jpg');
```

```
$('#input[type="checkbox"]').prop('checked', true);
```

```
const href = $('a').attr('href');
```

```
const isDisabled = $('button').prop('disabled');
```

Modifica del contenuto degli elementi

```
// Set the inner HTML of an element
$('div').html('<p>Hello, World!</p>');
```

```
// Get the inner HTML of an element
const html = $('div').html();
```

Inserimento di elementi

```
$('#ul').append('<li>Item</li>');
```

```
$('#ul').prepend('<li>Item</li>');
```

```
$('#li').before('<li>Item</li>');
```

```
$('#li').after('<li>Item</li>');
```

Rimozione di elementi

```
$('#li').remove();
```

```
$('#li').empty();
```

2.6 Un confronto tra le librerie

Dopo aver descritto i punti fondamentali delle librerie Selenium, Puppeteer e Cheerio, forniamo un confronto.

Selenium

Selenium è la più storica, nata nel 2006 e gode di una grande community e supporto su varie piattaforme e linguaggi di programmazione differenti.

Supporta innumerevoli browser (Google, Mozilla , Edge , Explorer).

Implementa il protocollo WebDriver, Grid, permettendo lo sviluppo di progetti di media grandezza.

Alcuni importanti limiti della libreria Selenium sono la gestione delle attese dovute ad alcuni problemi di sincronizzazione tra webdriver e browser. Selenium Bi-Di è un protocollo in via di sviluppo che in futuro risolverà questo problema.

Essendo una libreria a 3 livelli (webdriver, driver, browser), presenta una velocità di esecuzione più lenta rispetto alle altre librerie.

La versatilità di Selenium che consente di utilizzare vari script che utilizzano vari browser con diverse versioni e la sua grande diffusione la rendono la scelta più adatta in caso si vogliano effettuare test su vari siti con diversi browser.

Puppeteer

Puppeteer è più recente, è una libreria Node.js più specifica per il browser Chrome. Le API sono molto intuitive e l'apprendimento risulta più semplice rispetto a Selenium.

L'architettura di questa libreria presenta solo 2 livelli (il programma e il browser) e la comunicazione è realizzata dal protocollo Chrome Developers Tool. A livello di velocità di esecuzione è più veloce di Selenium.

Nel caso in cui l'interesse principale sia utilizzare il browser Chrome Puppeteer è la scelta migliore.

Cheerio

Cheerio è una libreria sviluppata sulla base di JQuery che esegue codice lato server-side per caricare e analizzare codice HTML.

Tra tutte le librerie analizzate, Cheerio è la più veloce.

È una libreria costruita per essere utilizzata esclusivamente nell'ambito del web scraping.

Un grosso limite di cheerio è che non ha funzionalità di effettuare parsing su javascript, una funzionalità molto utile dato che le SPA (single page application) ne fanno uso massivo.

Un vantaggio chiaro è la sua facilità e semplicità d'uso visto e considerato che la sintassi è la medesima di JQuery.

La velocità di esecuzione di Cheerio può essere significativa in caso le operazioni da effettuare siano molteplici in tempi limitati.

Indicatore	Selenium	Puppeteer	Cheerio
Caricamento della pagina	Tempo di risposta moderato, dipende dal browser scelto e dal sito. Occorre considerare che il flusso di lavoro si	Più veloce rispetto a Selenium. I livelli di lavoro sono 2 al posto dei 3 di Selenium	Non necessario visto che Cheerio non carica alcuna pagina web.

	componi di 3 livelli (browser,driver e webdriver) . In caso il driver o browser non siano installati, sarà necessario impiegare tempo per installarli e quindi i tempi di caricamento di allungheranno.		
Velocità di esecuzione	Più lenta rispetto alle altre librerie. Ogni comando necessita di una traduzione, ovvero il driver deve interpretare le direttive del webdriver per comandare il browser. Tutto ciò ha effetti sulla velocità di esecuzione.	Più veloce rispetto a Selenium visto l'utilizzo diretto delle API di Chrome. Un'ulteriore vantaggio è un layer in meno rispetto a Selenium.	Molto più veloce delle altre librerie.
Gestione degli errori	A causa dei problemi di sincronizzazione, la gestione degli errori può diventare molto difficile all'aumentare delle richieste. Questo problema in futuro sarà risolto con l'introduzione di Selenium BI-DI	La gestione risulta molto più comoda in quanto la sincronizzazione è effettuata in automatico dai comandi della libreria.	

2.7 Note finali

Dopo aver descritto le librerie occorre scegliere quale di esse sia la più adatta per lo sviluppo di un'applicativo di Web scraping.

Considerando tutto ciò che è stato detto fino ad ora il consiglio è quello di utilizzare comunque la libreria Selenium.

L'unico svantaggio come detto è la sua velocità se comparata alle altre librerie, questo è dovuto in parte al fatto che utilizzi un'architettura basata su 3 livelli, ma il problema più grande rimane la sincronizzazione tra browser e programma.

La velocità di Selenium può essere trascurata in caso la mole di dati da analizzare o scaricare via Web sia limitata.

Inoltre, nei prossimi anni il protocollo BI-DI (bidirectional) sarà ufficialmente implementato in Selenium risolvendo il problema di sincronizzazione e aumentando di conseguenza la velocità di esecuzione.

Oltre alla questione relativa alla velocità, il fatto che Selenium esista ormai da quasi due decenni offre la possibilità di utilizzare uno strumento solido.

Capitolo 3: Creazione prototipo

Nei precedenti 2 capitoli abbiamo fornito principalmente una descrizione teorica e architetturale riguardante il funzionamento generale dei programmi di Web scraping.

Obiettivo

In questo capitolo ci occuperemo di realizzare un'applicazione che faccia uso della libreria Selenium descritta nel precedente capitolo al fine di ottenere alcuni dati riguardanti i principali titoli di borsa.

Il codice completo è disponibile all'indirizzo <https://github.com/11THUNDER11/Tesi>

Nello specifico, le informazioni che si ritengono interessanti sono la quotazione (il prezzo) e vari indicatori finanziari come il volume di scambi, l'utile per azione e l'indice PE.

I risultati ottenuti verranno successivamente analizzati in maniera semplicistica per mettere in evidenza la fase successiva a quella del reperimento dati ovvero quella dell'analisi di questi ultimi.

Analizzeremo alcune possibili strategie di implementazione giustificando il motivo di alcune scelte fatte.

3.1 Scelta di un sito di riferimento

Il primo passo per reperire dei dati è la scelta del sito o di vari siti da visitare. Questo prototipo utilizzerà come risorsa Web il sito Yahoo Finance raggiungibile mediante il link <https://finance.yahoo.com/> il quale rende pubblici dati relativi alle quotazioni di una vasta gamma di titoli e indicatori di prezzo.

Il sito consente un facile accesso ai vari titoli come vedremo nei capitoli successivi e non presenta impedimenti di accesso dal punto di vista del rilevamento.

3.2 Analisi della struttura della pagina

In questa fase è importante capire dove si trovano i dati interessanti e se esiste la possibilità di modificare parte dell'url per accedere a diverse quotazioni. Forniamo un esempio con 2 titoli generici: Ferrari e Pirelli.

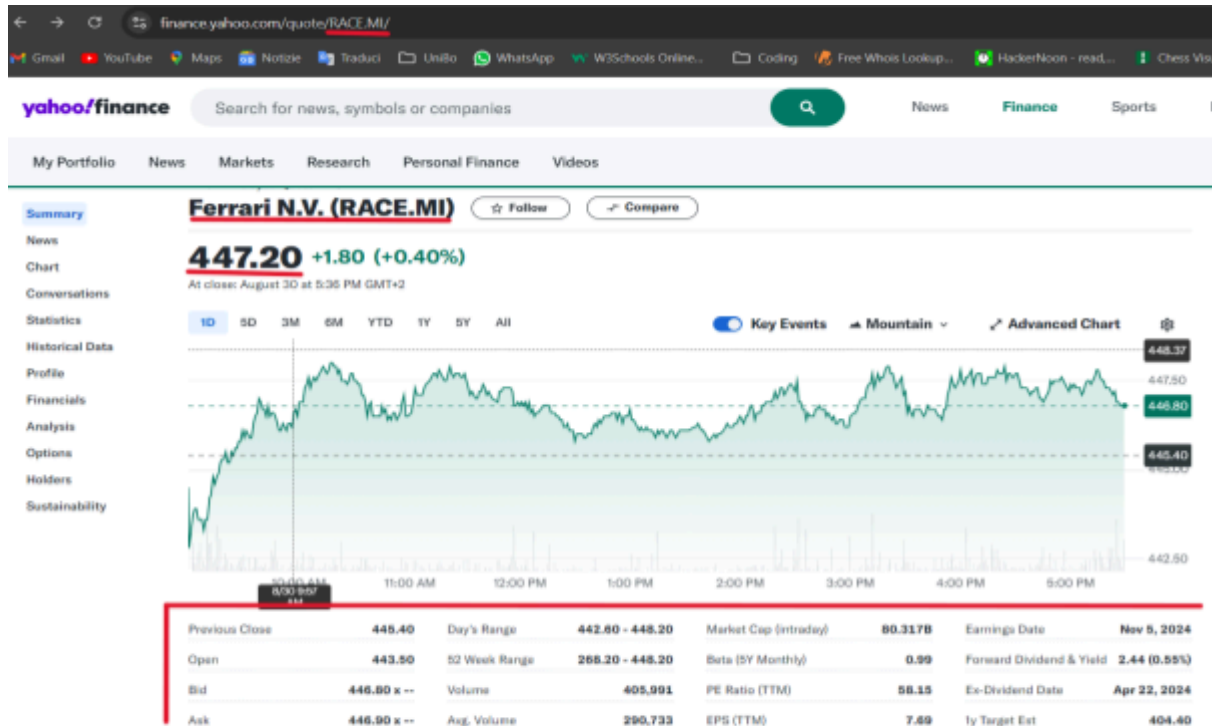


Figura 3.1 - Analisi pagina titolo Ferrari (ticker RACE.MI)

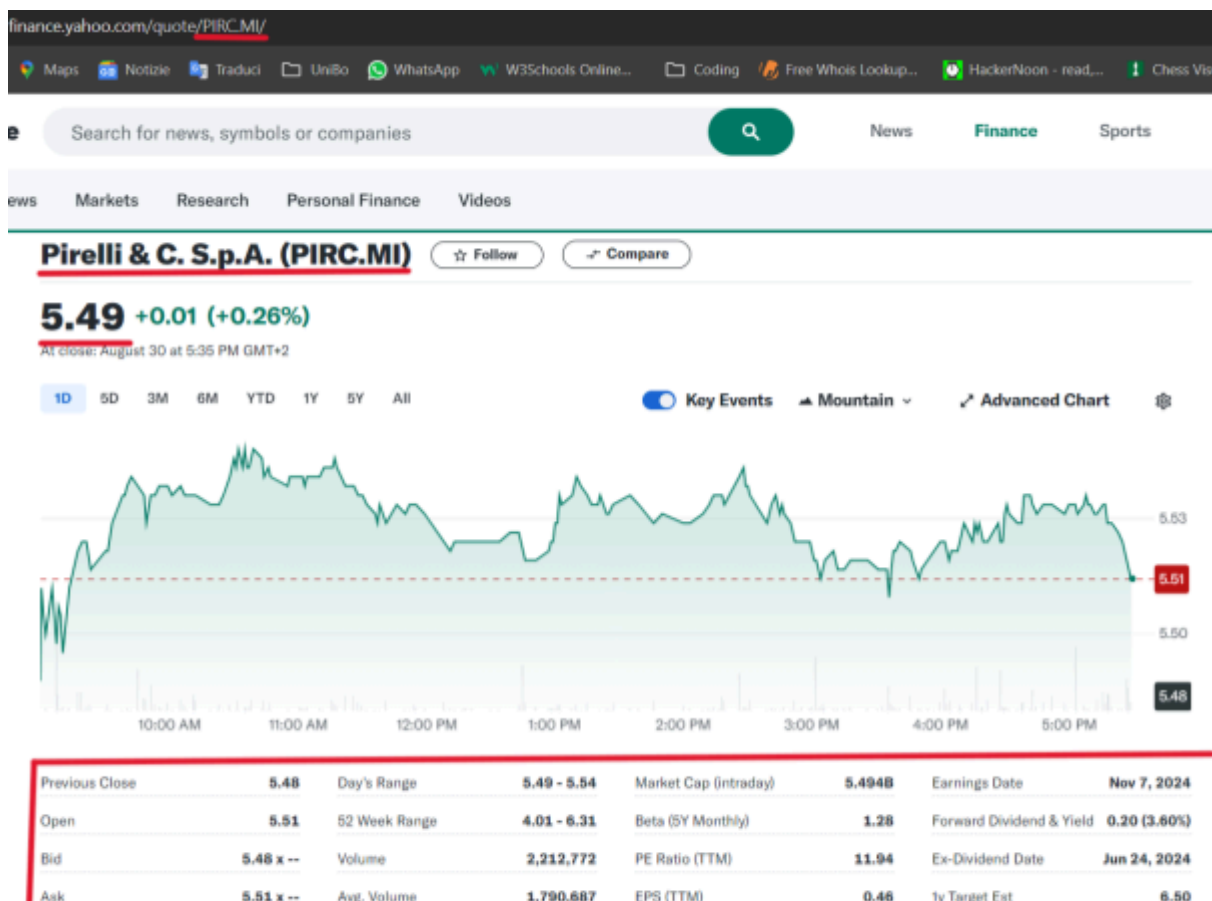


Figura 3.2 - Analisi pagina titolo Pirelli (ticker PIRC.MI)

Un'osservazione alquanto utile è la possibilità di cambiare l'ultima parte dell'url per accedere alle informazioni relative ad un altro titolo.

- <https://finance.yahoo.com/quote/RACE.MI/>
- <https://finance.yahoo.com/quote/PIRC.MI/>

Quindi in generale si potrà accedere ai vari titoli interrogando l'indirizzo [https://finance.yahoo.com/quote/\[TICKER\]](https://finance.yahoo.com/quote/[TICKER]) dove [TICKER] è l'identificativo univoco dell'azione specifica.

Questo può suggerire che i dati verranno visualizzati nella pagina seguendo una gerarchia di nomi di indirizzi e classi note.



Figura 3.3 - Dettaglio nome azienda titolo Pirelli



Figura 3.4 - Dettaglio nome azienda titolo Ferrari

È infatti presente un indirizzo di classe comune "yf-3a2v0c" per riferire il nome dell'azienda.

Dopo aver verificato che i dati possono essere ottenuti interrogando indirizzi noti, occorre ottenere quelli relativi al prezzo di quotazione e ai dati aziendali.

Il prezzo si trova all'indirizzo di classe "livePrice yf-mgkamr", mentre quello per i dati aziendali è "yf-tx3nkj".

Ferrari N.V. (RACE.MI)

☆ Follow

↔ Compare

447.20 +1.80 (+0.40%)

```
<div class="price" data-cy="price">
  <div class="livePrice yf-ngkwn" data-
    symbol="RACE.MI" data-testid="qsp-price" data-
    field="regularMarketPrice" data-trend="none" data-
    pricehint="2" data-values="447.2" active>
    <span>447.20</span> == $0
  </div>
</div>
```

Figura 3.5 - Dettaglio del prezzo

```
<ul style="--rows-sm: 8; --rows-md: 6; --rows: 4; -
  -max-col: 4" class="yf-tx3nkj">...</ul> grid == $0
```

Figura 3.6 - Dettagli indicatori

Arrivati a questo punto teniamo a mente gli indirizzi che abbiamo trovato perché occorreranno in fase di realizzazione dello scraper.

3.3 Costruzione di uno scraper

In questa fase occorre fare alcune considerazioni relative alla struttura interna che risulterà dalle scelte che verranno prese.

Uno scraper per uno o più url?

Occorre considerare che un singolo scraper collegato ad un unico url scaricherà i dati più velocemente e reagirà ai cambiamenti relativi ai dati contenuti nella pagina.

Immaginiamo di essere interessati ad una generica sorgente di dati che sono soggetti a cambiamenti frequenti.

In questo caso la scelta più logica sarebbe utilizzare un singolo scraper per ogni url interessato.

Utilizzare un singolo scraper per ogni url ha qualche importante limitazione, non è scalabile all'aumentare degli url per limiti di memoria e in caso di cpu "lente" la creazione di molti driver può dare luogo a lunghi colli di bottiglia.

In Selenium questo si traduce nella creazione di un'istanza di driver per ogni url da visitare il che, oltre a produrre un significativo overhead sia nell'apertura che nella chiusura del driver, aumenta lo spazio di memoria occupato.

La scelta di utilizzare un singolo scraper per diversi url è vantaggiosa dal punto di vista dell'occupazione di memoria mentre invece la velocità subisce un rallentamento in caso la coda degli url da visitare sia numerosa.

Ogni volta che una pagina viene visitata, è necessario adottare tecniche di attesa per cercare di aspettare che i vari elementi siano presenti. Anche pochi millisecondi possono diventare rilevanti al crescere degli url interessati.

In più, essendo ora la visita ai vari url sequenziale, può accadere che l'accesso ad una pagina Web fallisca, oppure che i dati non vengano trovati, in questo caso occorrerà istruire il programma per gestire accuratamente alcune condizioni di errore.

Nel caso dell'applicazione, la velocità di accesso ai dati non è molto rilevante. Ciò che importa maggiormente sono i valori aziendali contenuti nella lista a fondo pagina del sito, difficilmente questi dati subiranno modifiche frequenti e per gli scopi dell'applicazione l'accesso in tempo reale a queste informazioni non è vincolante nelle scelte progettuali.

Non vogliamo però che i tempi di risposta diventino eccessivi, adottiamo quindi una soluzione ibrida, ovvero utilizziamo più scraper, ognuno di essi contiene una lista di url da scaricare.

Il numero di scraper può variare da uno a circa dieci, il motivo è che la gestione contemporanea di più di 10 driver può risultare lenta da parte della CPU.

Gli url appartengono allo stesso tipo di pagina?

Nel caso che uno scraper ricevesse una lista di url da visitare, occorre capire se i vari url riferiscano ad una pagina che presenta la medesima struttura.

Nel caso della nostra applicazione la pagina presenta la stessa struttura con i dati che variano modificando la parte finale dell'url, perciò come abbiamo mostrato gli elementi per localizzare i dati non cambiano.

Diversamente, se la lista fosse formata da pagine con una struttura diversa, occorre specificare quali elementi scaricare per ciascun url dato che la locazione degli elementi cambia.

Una soluzione potrebbe essere l'accostamento dell' url del sito ad una struttura dati di tipo elemento.

[url, List<Element>] in cui Element è formato da
[IDTYPE, name] dove IDTYPE rappresenta l'identificativo di classe o di indirizzo che referencia l'elemento e può assumere i valori ID o CLASS.

3.3.1 Architettura dell'applicazione

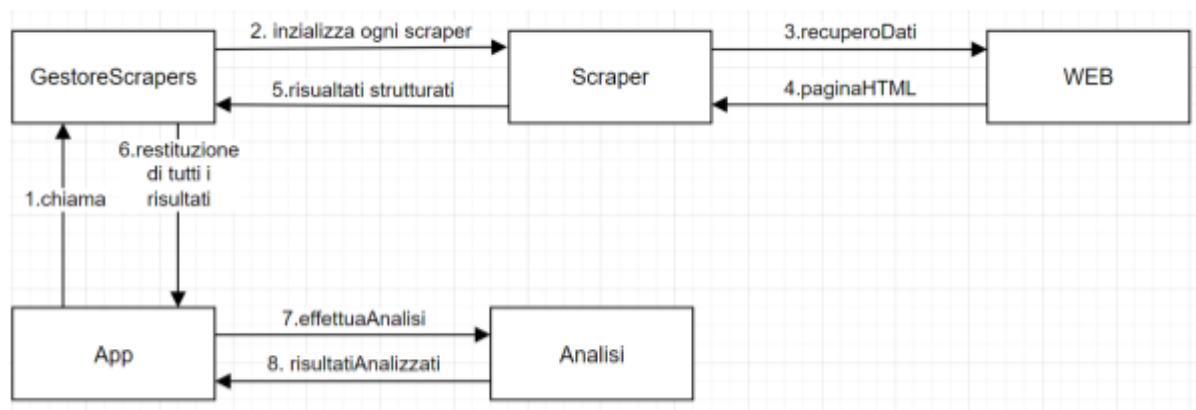


Figura 3.7 - Architettura del prototipo

L'architettura scelta presenta vari livelli.

1. **App** : componente che ha il ruolo di gestire ad alto livello il funzionamento dell'applicazione, stabilisce quando è il momento di ricercare i dati, con quale frequenza effettuare le richieste e con quanti thread. Salva i dati per una successiva analisi e invia messaggi alla parte front end quando riceve informazioni aggiornate.
2. **Gestore Scrapers** : componente responsabile della gestione dei vari scraper relativi ai vari url ricevuti dall'applicazione. Stabilisce il momento per inizializzare i vari driver dei vari scraper e il momento per scaricare le varie informazioni.
Quando tutti i dati sono stati ottenuti segnala all'applicazione la corretta riuscita, in caso di errori segnala l'impossibilità di ottenere i dati ma tutto ciò è trasparente all'utente.
3. **Scraper** : componente di basso livello che contiene le informazioni necessarie per localizzare gli elementi di una determinata pagina Web. I vari identificativi degli ipertesti sono quelli sopra citati al momento dell'ispezione del sito.
4. **Analizzatore** : componente che riceve i dati scaricati e sulla base di alcuni parametri effettua un'analisi semplicistica. Tra i vari dati scaricati è presente un indicatore finanziario, il Price Earnings Ratio, che misura quanto gli investitori sono disposti a pagare per una singola unità di utile. Sulla base di questo solo indicatore viene dato un giudizio sul titolo, sotto il

livello di 20 l'azione è considerata a buon mercato, sopra i 25 costosa mentre tra 20 e 25 il prezzo è bilanciato.

Questo indicatore può non essere presente perché può essere calcolato solo se l'azienda ha generato utile per corso del trimestre/anno precedente, in questo caso non viene fornita una valutazione.

3.3.2 Creazione del driver

Il componente Scraper, il più importante dell'applicazione si compone di due metodi principali:

- inizializzazione del driver Selenium
- apertura della pagina Web indicata

Queste due fasi rappresentano i passi descritti nel secondo capitolo.

1. La prima fase, quella di inizializzazione, ha lo scopo di creare un nuovo driver che sarà poi comandato per ricercare informazioni sul Web.
2. Creiamo quindi un driver per il browser Chrome (la scelta è arbitraria) con varie opzioni, ad esempio attivando la modalità headless per non vedere visivamente come stia operando il browser comandato dal programma.

In questo metodo si fa uso del componente Selenium Driver Manager (descritto nel capitolo 2) ma non è necessario scaricare nulla in quanto la versione di Chrome è già presente sul dispositivo locale.

```
// Metodo per inizializzare il driver del browser
async initDriver() {
  if(this.#seleniumChromeDriver == null){

    //Impostiamo le varie opzioni
    options.addArguments("--disable-search-engine-choice-screen");
    options.addArguments("--headless=new");
    options.addArguments('--log-level=2');
    options.addArguments('--ignore-certificate-errors');
    options.addArguments('--ignore-ssl-errors');
    options.addArguments('--ignore-certificate-errors-spki-list');

    // Creazione di una nuova istanza del driver di Chrome
    this.#seleniumChromeDriver = await new Builder().forBrowser(Browser.CHROME)
      .setChromeOptions(options)
      .build();

    this.#state = scraperState.BUILD;
  }
}
```

Figura 3.8 - Creazione del driver Selenium

Alla fine del metodo impostiamo lo stato del driver in modo che possa essere rilevato dai successivi metodi.

3.3.3 Apertura della pagina

Successivamente alla creazione del driver, il prossimo passo è raggiungere l'url indicato e scaricare i dati a cui si è interessati.

```
// Metodo per aprire la pagina web
async openPage(url) {
  return new Promise(async (resolve, reject) => {
    try {
      await this.#seleniumChromeDriver.get(url);
      await this.#seleniumChromeDriver.manage().setTimeouts({implicit: 1000});
      //Rimozione banner cookie
      if(!this.#removed){
        let rejectCookieElement = await this.#seleniumChromeDriver.findElement(By.name('reject'));
        await rejectCookieElement.click();
        this.#removed = true;
      }

      let companyName = await this.#seleniumChromeDriver.findElement(By.className('yf-3a2v0c'));
      let name = await companyName.getText();

      let priceElement = await this.#seleniumChromeDriver.findElement(By.className('livePrice'));

      let priceValue = await priceElement.getText(); // Ottiene il testo dell'elemento

      let stockIndicatorsElement = await this.#seleniumChromeDriver.findElement(By.className('yf-tx3nkj'));
      let rawValues = await stockIndicatorsElement.getText();
    }
  });
}
```

Figura 3.9 - Apertura browser e navigazione tra le varie pagine

Questo metodo viene chiamato solo successivamente all'effettiva creazione del driver Selenium, se la creazione precedente è fallita questo metodo non viene mai raggiunto.

Il metodo indicato viene chiamato tante volte quanti sono gli indirizzi dei titoli da scaricare.

Occorre evidenziare che la localizzazione degli elementi è statica, in quanto come descritto in precedenza l'applicazione scarica i dati da una singola pagina Web che ha la medesima struttura.

- la prima istruzione raggiunge l'url interessato mentre la seconda impone una strategia di attesa implicita, tutti i localizzatori avranno lo stesso limite di attesa, in questo caso un secondo.
- successivamente occorre rimuovere il banner relativo all'accettazione/rifiuto dei cookie che appare solo la prima volta che si accede al sito, si localizza il pulsante "Rifiuta" e si invia un segnale al driver per simulare l'evento di click su tale bottone.

Le istruzioni che seguono hanno lo scopo di localizzare e scaricare gli elementi importanti agli indirizzi prima elencati.

3.3.4 Gestione dei risultati

Una volta terminata la fase di scaricamento, i dati vengono restituiti all'applicazione e ne viene effettuata una semplice analisi per poi essere salvata su un file temporaneo.

```
"Company name": "Intel Corporation (1INTC.MI)",  
"date": "2/9/2024",  
"time": "11:27:37",  
"ticker": "1INTC.MI",  
"price": "19.70",  
"Previous Close": "19.82",  
"Open": "20.00",  
"Bid": "19.70 x --",  
"Ask": "19.78 x --",  
"Day's Range": "19.68 - 20.00",  
"52 Week Range": "17.20 - 46.30",  
"Volume": "7,369",  
"Avg. Volume": "26,712",  
"Market Cap (intraday)": "84.649B",  
"Beta (5Y Monthly)": "1.05",  
"PE Ratio (TTM)": "89.55",  
"EPS (TTM)": "0.22",  
"Earnings Date": "Oct 24, 2024 - Oct 28, 2024",  
"Forward Dividend & Yield": "0.46 (2.31%)",  
"Ex-Dividend Date": "Aug 7, 2024",  
"1y Target Est": "--",  
"valuation": "EXPENSIVE"
```

Figura 3.10 - Risultati scaricati per ciascun titolo

I dati vengono poi inviati ad una pagina React mediante collegamento tramite WebSocket, questa parte ha il solo scopo di mostrare all'utente i dati scaricati.

3.4 Una nota sulla frequenza delle richieste

L'applicazione naviga tra le varie pagine Web con una frequenza di circa un minuto in caso la ricerca precedente sia conclusa. È possibile infatti che lo scaricamento possa durare anche più di un minuto per vari problemi di rete o collegamento, in questo caso le varie richieste verranno effettuate solo il minuto successivo.

Occorre tenere a mente che troppe richieste contemporanee e a poca distanza temporale possono venire bloccate dal sito in quanto rilevate come possibile attacco informatico.

3.5 Alcuni parametri di performance

Facciamo un paio di test per misurare le performance delle librerie Selenium e Puppeteer.

3.5.1 *Test locale di comparazione Selenium-Puppeteer*

I test sono stati effettuati sulla macchina locale, la quale dispone delle seguenti caratteristiche :

- CPU : AMD RAYZEN 7 5700U with Radeon Graphics, 1801 Mhz, 8 core, 16 processori logici
- RAM : 16 GB
- Sistema operativo : Windows 10
- Modello : HP Pavilion Laptop 15
- Architettura : 64 bit
- Scheda di rete : Realtek RTL8821CE 802.11ac PCIe Adapter

Gli script utilizzati per effettuare le misure sono disponibili al sito <https://github.com/11THUNDER11/Tesi/tree/main/test> , si è utilizzata la stessa logica applicata nello scraper dell'applicazione con la differenza che i dati da reperire questa volta sono limitati al solo nome dell'azienda.

Di seguito i risultati ottenuti che vengono visualizzati in output alla fine delle prove.

Viene utilizzato un singolo scraper per scaricare 30 titoli diversi, questo processo viene ripetuto per 20 volte per ottenere dati più precisi.

SELENIUM	PUPPETEER
43.12	46.97
42.79	47.18
40.06	49.79
40.84	41.33
49.77	45.74
42.66	47.02
42.51	47.08
45.45	58.60
44.60	44.05
40.06	46.90
40.16	47.71
43.18	50.04
40.65	52.88
42.78	45.46
42.24	45.85
44.44	46.18
55.21	48.06
49.83	52.07
49.86	52.30
44.23	42.60

Una media dei risultati è mostrata nella seguente tabella

LIBRERIA	SELENIUM	PUPPETEER
TEMPO TOTALE MEDIO	44.22 secondi	47.89 secondi
TEMPO PER TITOLO	1,474 secondi	1,59 secondi
% CPU UTILIZZATA	25 %	15 %
MEMORIA UTILIZZATA	500 MB	340 MB

Tabella 3.11 - Comparazione risultati Selenium - Puppeteer

Le statistiche sono calcolate nel momento in cui il banner dei cookie viene rimosso così da evitare di falsare i numeri a causa dei differenti tempi di apertura dei driver.

- *Velocità*

Nonostante Selenium sia formato tra 3 livelli architetturali, invece che i soli 2 di Puppeteer (si veda il capitolo 2), non sembrano esserci sostanziali differenze di velocità di reperimento.

Una possibile spiegazione può essere la maggior velocità di Selenium a localizzare gli elementi della pagina che compensa l'uso di un livello aggiuntivo.

- *% CPU*

Una sostanziale differenza si presenta nell'uso della CPU, Selenium utilizza quasi il doppio del processore rispetto a Puppeteer.

Selenium, essendo formato da tre layer, presenta un maggior numero di operazioni da effettuare.

- *Memoria utilizzata*

Dal punto di vista della memoria, sembra che Selenium occupi molto più spazio rispetto a Puppeteer. Il driver utilizzato da Selenium risulta più ingombrante rispetto a quello di Puppeteer. Questo può suggerire di utilizzare un numero limitato di driver nelle applicazioni di scraping per non rischiare di riempire la memoria fino a livelli pericolosi.

3.5.2 Una considerazione emersa in fase di test

Occorre però fare una considerazione importante che non emerge dalle statistiche. Sembra che Puppeteer abbia problemi nella fase di rimozione del banner dei cookie, questo lo si può osservare svolgendo i test. Infatti può capitare che il bottone “Rifiuta” non sia presente all’interno della pagina al momento in cui è necessario cliccarlo.

Risulta quindi che la fase di apertura e inizializzazione del driver di Puppeteer sia più lunga rispetto a quella di Selenium.

3.5.3 Un'analisi con più driver

Effettuiamo ora alcuni test per valutare il numero ottimale di driver da utilizzare. Per comodità utilizzeremo solo la libreria Selenium utilizzata nel prototipo sopra mostrato.

Si parte da una base di 30 pagine relative a diversi titoli azionari, questi titoli verranno ripartiti equamente in base al numero di driver che il test utilizzerà.

Per una maggiore affidabilità dei risultati verranno condotti 20 ripetizioni per ogni driver.

Dopo una serie di tentativi si è scelto di riportare i risultati ottenuti fino a 5 driver.

La fase di accettazione dei cookie presenta un problema non trascurabile. Dopo aver accettato o rifiutato i cookie, si viene indirizzati verso una pagina del sito temporanea e solo successivamente sulla pagina che contiene il nome dell’azienda che ci interessa.

In caso la pagina importante non venga caricata in tempo il programma non sarà più sincronizzato con il browser e restituirà un errore.

Questo problema non era presente all’inizio di questo documento perché non erano ancora stati condotti test intensivi con una grande frequenza di richieste per volta. Un motivo può riguardare il fatto che Yahoo - Finance rilevi la presenza di Selenium nel momento in cui la richiesta viene effettuata e successivamente reindirizzi la richiesta verso una pagina diversa da quella iniziale.

Perciò l'utilizzo di valori maggiori a 5 comporta un gran numero di situazioni d'errore sulla macchina locale sia dovute all'eccessiva attesa del programma.

I dati relativi al tempo impiegato riguardano il tempo che impiega il driver a scaricare i dati partendo dal primo ticker fino all'ultimo che gli è stato assegnato, non comprendendo la fase di inizializzazione del driver.

Nel caso del nostro prototipo, per raggiungere le informazioni che si ritengono rilevanti, è necessario accettare i cookie istruendo Selenium a cliccare sull'elemento corretto. Quindi la fase che si ritiene più importante riguarda l'effettiva navigazione tra i vari titoli.

I test sono stati effettuati utilizzando la modalità headless, ovvero senza il dover vedere un browser visualizzato a schermo, quindi in caso venissero effettuati test in modalità headful (visualizzando il browser) i valori indicati potrebbero salire.

Numero drivers	1	2	3	4	5
CPU (%)	15-25	25-40	30-35	32-40	45-50
RAM (MB)	220-300	450-650	700-750	800-950	900-1024
Tempo (secondi)	42.06	26.22	26.78	24.20	22.87

Figura 3.12 - Prestazioni di Selenium con driver multipli

Secondo i dati risultanti, si nota che il passaggio da 2 driver a 3 non presenta una significativa riduzione del tempo impiegato per lo scaricamento.

Come previsto in precedenza, all'aumentare dei driver, cresce lo spazio di memoria occupato totale di circa 200-250 MB per volta mentre per quanto riguarda la CPU si osserva un incremento medio del 10%.

A fronte di questi risultati e delle precedenti osservazioni riguardanti le situazioni di errore, possiamo concludere che il numero ottimale di driver da utilizzare si aggira sui 2 driver.

3.5.4 *Una nota su Cheerio*

A differenza di Selenium e Puppeteer, Cheerio non necessita di un browser locale per reperire risorse, essendo un parser di contenuti html l'unica cosa di cui necessita è codice HTML scaricato dalla rete.

Comparare Cheerio mediante i test effettuati sopra non sarebbe corretto, in quanto il funzionamento di questa libreria è assai diverso e molto più limitato.

- l'accettazione o il rifiuto dei cookie è necessario al momento del collegamento al sito Yahoo Finance ma Cheerio non operando con un browser non rende il passaggio di rimozione del banner cookie semplice.
- un successivo problema all'utilizzo di librerie sopra citate è che esse sono ben note ai siti e sono soggette a probabili blocchi, rendendo così la navigazione impossibile.
- a livello di velocità Cheerio sarebbe sicuramente la libreria più veloce, sempre che non vengano effettuate troppe richieste per volta. Il fatto che non possa interagire con gli elementi della pagina rappresenta un grosso limite.

Conclusioni

Nel corso di questa tesi è stato affrontato il tema del Web scraping, ovvero l'estrazione automatizzata di dati da siti Web attraverso l'uso di script e strumenti software.

Questo processo rappresenta una pratica sempre più diffusa nel campo dell'informatica e dei big data che consente di reperire grandi quantità di informazioni disponibili pubblicamente su Internet.

In particolare, si è prestata attenzione all'utilizzo di alcune librerie software open-source che facilitano la realizzazione di applicativi per il Web scraping, consentendo a sviluppatori e analisti di estrarre e strutturare dati per diversi scopi.

L'implementazione di un piccolo applicativo ha dimostrato la fattibilità tecnica del processo di Web scraping evidenziando che, anche senza grandi risorse hardware o software avanzati, è possibile raccogliere una discreta quantità di dati in tempi ragionevolmente brevi.

Nello specifico, è stata utilizzata la libreria Selenium che, sebbene non sia tra le più performanti dal punto di vista della velocità di esecuzione, si distingue per la sua capacità di simulare un browser reale e di interagire con le pagine Web in modo simile a un utente umano.

Vantaggi e limitazioni dell'approccio con Selenium

Selenium si dimostra particolarmente utile quando si ha a che fare con siti Web complessi, che fanno uso di JavaScript, AJAX o altre tecnologie di rendering dinamico del contenuto. Infatti, molte delle pagine Web moderne non caricano tutti i dati immediatamente, ma li caricano in modo asincrono dopo l'interazione dell'utente con la pagina stessa. In questo contesto, Selenium offre un vantaggio significativo rispetto ad altre librerie che non prevedono l'uso di un browser, in quanto permette di interagire dinamicamente con gli elementi della pagina eseguendo script JavaScript, compilando moduli, cliccando pulsanti e navigando in modo efficace tra le diverse sezioni di un sito. Questo risulta di particolare importanza nel momento in cui il programma venga reindirizzato ad una pagina di accettazione dei cookie.

Tuttavia, nonostante questi vantaggi, Selenium presenta alcune limitazioni principalmente in termini di prestazioni.

Poiché l'esecuzione del codice implica l'apertura di un vero e proprio browser, ciò richiede un utilizzo significativo di risorse hardware tra cui una grande occupazione di memoria e una latenza maggiore rispetto a strumenti di scraping più leggeri, che non prevedono l'uso di un browser. Questi ultimi, essendo basati su richieste HTTP

dirette, risultano molto più rapidi nel recuperare dati statici da pagine Web che non richiedono l'esecuzione di codice client-side.

Ulteriori approfondimenti: coordinazione e sistemi anti-bot

Un aspetto interessante da approfondire riguarda la possibilità di coordinare più scraper simultaneamente su differenti siti. Questo approccio permette di scalare il processo di raccolta dati, garantendo un'estrazione più rapida e distribuita, riducendo al contempo il rischio di essere bloccati da un singolo server.

Contestualmente, occorre considerare l'esistenza di sistemi anti-bot, utilizzati da molti siti per bloccare l'accesso automatizzato ai loro contenuti. Questi sistemi, basati su tecnologie come CAPTCHA, rilevamento di comportamenti anomali e tracciamento di richieste provenienti dallo stesso indirizzo IP, rappresentano una sfida per chi si occupa di scraping. Approfondire il funzionamento di tali sistemi può aiutare gli sviluppatori a progettare scraper che riescano a operare senza essere rilevati o bloccati. Alcune tecniche per eludere questi sistemi includono l'uso di proxy che utilizzano più indirizzi IP in rotazione e il rallentamento deliberato delle richieste per imitare un comportamento umano.

Aspetti legali e normativi del Web scraping

Un ulteriore ambito che meriterebbe di essere esplorato riguarda la normativa vigente in materia di Web scraping. Nel capitolo introduttivo di questa tesi sono stati solo accennati i rischi giuridici associati all'estrazione automatica di dati da siti Web, ma è fondamentale sottolineare come questo sia un tema di crescente rilevanza, soprattutto in relazione a questioni come il diritto d'autore e la protezione dei dati personali.

Molti siti inseriscono nelle proprie condizioni d'uso delle clausole che vietano esplicitamente l'estrazione automatica mediante dei programmi di scraping, riservando il diritto di intraprendere azioni legali nei confronti di chi violi tali condizioni. Tuttavia, la situazione legale varia notevolmente da paese a paese, e non sempre è chiaro quale sia la linea di confine tra scraping legittimo e violazione delle norme.

Infine, merita attenzione anche la General Data Protection Regulation (GDPR), la normativa europea sulla protezione dei dati personali, che impone restrizioni molto severe sull'uso e l'elaborazione di dati sensibili. Anche se non sempre i dati raccolti sono personali, in alcuni casi potrebbe comportare la raccolta di informazioni identificabili, e questo potrebbe esporre chi lo pratica a rischi giuridici rilevanti.

In conclusione, il Web scraping rappresenta una tecnica potente e versatile, ma allo stesso tempo complessa, sia dal punto di vista tecnico che normativo. Gli

sviluppatori che decidono di utilizzarla dovrebbero essere consapevoli delle sfide tecniche e legali che essa comporta, e dovrebbero valutare attentamente i rischi e le opportunità legate all'uso di tale tecnologia.

Bibliografia

1. “Informazione italiani, più social ma senza fidarsi troppo” - <https://www.ilsole24ore.com/art/l-informazione-italiani-piu-social-ma-senza-fidarsi-troppo-AEXRgt2D>
2. “I bot rappresentano il 42% del traffico Web complessivo e i due terzi sono dannosi”
<https://www.tomshw.it/business/i-bot-rappresentano-il-42-del-traffico-web-complessivo-e-i-due-terzi-sono-dannosi>
3. “Google IA scraping as fair use” - <https://www.tomshardware.com/news/google-ai-scraping-as-fair-use>
4. Selenium - <https://www.selenium.dev/>
5. Puppeteer - <https://pptr.dev/>
6. Cheerio - <https://cheerio.js.org/>
7. Progetto scraper - <https://github.com/11THUNDER11/Tesi>